

RESTAURANT COMMERCE & REWARDS PLATFORM

DAMRU

Developer Technical Documentation
Website + Backend + React Native

Architecture + Data Models + APIs + Reward Engines + Operations

Version	Documentation Date	Scope
1.0	7 August 2026	Current repository implementation

Table of Contents

Thirty sections covering architecture, data, APIs, reward engines, and operations.

01 Project Overview

02 Technology Stack

03 Repository Structure

04 Runtime Architecture

05 Authentication Architecture

06 Core Data Models

07 API Inventory

08 Validation and Error Handling

09 Damru Reward Architecture

10 Reward Event Map

11 Daily Streaks

12 Achievements

13 Missions

14 Referrals

15 Loyalty Tiers

16 Checkout and Order Lifecycle

17 Website Architecture

18 Mobile Architecture

19 Admin Architecture and Permissions

20 Email and Scheduled Work

21 Security Properties

22 Environment Variables

23 Local Development

24 Build and Deployment

25 Testing and Known Issues

26 Common Extension Workflows

27 Troubleshooting

28 Production Checklist

29 Developer Onboarding Checklist

01 Project Overview

What this repository is, and how its pieces fit together.

Damru is a TypeScript monorepository-style project containing a Next.js website, a Next.js route-handler backend, an administration interface, an Expo/React Native mobile application, Mongoose models, and business services. MongoDB is the persistent store.

The website and mobile client are two independent front doors that share the same backend endpoints. The backend owns authentication, commerce validation, reward eligibility, progress tracking, and redemption — none of that logic is duplicated or re-implemented on either client.

Reading this document

Each section describes confirmed repository structure and behavior as implemented today. Where a detail could not be confirmed from the codebase or committed configuration, it is called out explicitly rather than assumed.

02 Technology Stack

Core languages, frameworks, and libraries in the current repository.

Layer	Technology	Version	Purpose
Web / backend	Next.js	16.2.4	App Router, server actions, route handlers
Web UI	React / React DOM	19.2.4	Website and admin rendering
Language	TypeScript	5.x root / 6.0 mobile	Static typing
Database	MongoDB + Mongoose	Mongoose 9.5.0	Persistence and schemas
Authentication	NextAuth beta + custom customer sessions	5.0.0-beta.31	Admin and customer authentication patterns
Mobile	Expo	57.0.10	React Native application runtime
Mobile UI	React Native	0.86.2	Native screens
Mobile routing	Expo Router	57.0.10	File-based navigation
Mobile data	TanStack Query	5.101.4	Query caching and retry policy
Email	Nodemailer / Resend	7.0.13 / 6.18.1	Transactional messages
Passwords	bcryptjs	3.0.3	Password hashing
Tokens	jsonwebtoken	9.0.3	Signed token support

Resend is installed but not the live email path

Nodemailer/SMTP is the transactional email path actually used by `lib/email.ts`. The Resend dependency is present in `package.json` but not wired into the send path — confirm before relying on it.

03 Repository Structure

Top-level layout of the maintained source tree.

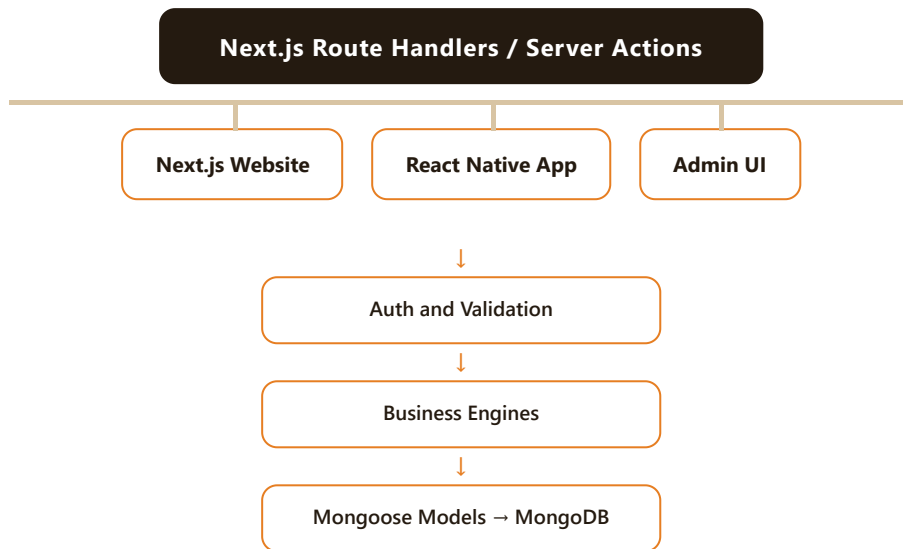
damru/

- ├─ app/ Next.js pages, admin, APIs, server actions
- ├─ components/ Website and admin components
- ├─ lib/ Database, auth, rewards, and business services
- ├─ models/ Mongoose models
- ├─ public/ Static website assets
- ├─ scripts/ Seeds, schedulers, reconciliation utilities
- ├─ styles/ Website stylesheets
- ├─ docs/source/ Documentation sources
- ├─ mobile-app/src/app/ Expo Router screens
- ├─ mobile-app/src/components/ Mobile UI
- ├─ mobile-app/src/lib/ API/query/analytics helpers
- ├─ mobile-app/src/services/ Backend API clients
- ├─ mobile-app/src/types/ Mobile response types
- ├─ auth.ts Admin authentication integration
- ├─ proxy.ts Request proxy/middleware behaviour
- ├─ next.config.ts, package.json
- └─ tsconfig.json

Generated directories such as `node_modules`, `.next`, native build output, and Git internals are not part of the maintained source inventory and are not scanned or documented here.

04 Runtime Architecture

How the website, mobile app, and admin UI reach the same backend.



The mobile app calls the same `/api/...` surface exposed to the website — there is no separate mobile-only backend. Server-only modules connect to MongoDB through `lib/mongodb.ts`. Business engines sit between event sources (route handlers, server actions) and models so reward rules are defined once and never duplicated in either client.

05 Authentication Architecture

Two separate identity systems: customer sessions and admin auth.

Customer endpoints cover registration, OTP sending and verification, login, logout, current-user lookup/update, and password reset. Customer route handlers obtain the authenticated user through the established customer-session helper and return `401` when absent. Admin authentication uses the project's NextAuth configuration and permission helpers.

Password and verification flows must remain server-owned

Secrets must never be returned to clients. OTP values are handled entirely server-side as part of the verification flow.

Relevant routes

`/api/user/register` · `/api/user/login` · `/api/user/logout` · `/api/user/me` · `/api/user/send-otp` · `/api/user/verify-otp` ·
`/api/user/reset-password` · `/api/auth/[...nextauth]`

06 Core Data Models

Every Mongoose model in the repository and what it's responsible for.

Model	Responsibility	Important constraints / indexes
User	Identity, profile, wallet totals, occasions, streak, referral, loyalty state	Unique email; sparse unique referral code; loyalty tier reference
Order	Items, totals, payment and fulfilment state	Unique human order ID; user/status/date indexes
Cart	Customer cart state	Customer association
Menuitem / Category	Sellable menu and grouping	Application-level validation
Coupon	Public/private discounts and reward coupons	Code and customer eligibility constraints
DamruTransaction	Financial reward ledger	Unique idempotency key; user/date index
RewardRule	Configured base rewards	Category configuration
DamruConfig	Redemption and legacy reward settings	Singleton-style access service
DailyStreakConfig	Daily reward sequence and recovery policy	Normalised configuration
Achievement / UserAchievement	Achievement definitions and per-user state	Definition code and user progress uniqueness
Mission / UserMission / MissionEvent	Mission definitions, user state, event deduplication	Deterministic period/event identity
ReferralConfig / Referral	Referral policy and lifecycle	Referral identity/status controls
LoyaltyTier	Configurable tier ladder	Unique code and rank; validated ranges
Address	Saved customer delivery addresses	User ownership
Admin	Staff identity, role, permissions	Administrative authentication fields
Branch / Table	Restaurant locations and table operations	Operational identifiers
Reservation / BanquetBooking	Booking requests	Lifecycle fields
Blog / BlogCategory / BlogComment	Content publishing	Slug/category relationships
GalleryTab / SiteSettings	Site-managed content	Configuration records
Complaint	Customer support cases	Customer ownership/status

07 API Inventory

Every route group, organized by domain.

Customer and commerce

Method family	Endpoint	Purpose
GET/POST/PATCH/DELETE	<code>/api/address</code>	Saved-address operations
GET/POST/PATCH/DELETE	<code>/api/cart</code> , <code>/api/cart/item</code>	Cart state and items
GET/POST	<code>/api/orders</code>	Customer order list and creation
GET	<code>/api/menu</code> , <code>/api/home-menu</code> , <code>/api/menu/suggestions</code>	Menu discovery
GET	<code>/api/search</code>	Site search
GET/POST	<code>/api/coupons</code>	Coupon validation/listing patterns
GET/POST/PATCH	<code>/api/reservations</code> , <code>/api/reservations/[id]</code>	Reservations
GET/POST	<code>/api/banquet-bookings</code>	Banquet enquiries
GET/POST/PATCH	<code>/api/complaints</code> , <code>/api/complaints/[id]</code>	Support complaints

Content

`/api/branches` · `/api/blogs/[slug]` · `/api/blog-search` · `/api/blog-categories` · `/api/blog-comments` · `/api/gallery` · `/api/homepage-blogs` · `/api/upload` support published content and media workflows.

Customer rewards

Method	Endpoint	Purpose
GET	<code>/api/rewards/dashboard</code>	Lightweight wallet and programme summaries
GET	<code>/api/rewards/history</code>	Paginated ledger history
GET	<code>/api/rewards/coupons</code>	Eligible reward coupons
GET	<code>/api/rewards/upcoming</code>	Occasion and legacy summary data
POST	<code>/api/rewards/redeem</code>	Order-bound Damru redemption
GET	<code>/api/rewards/achievements</code>	Authoritative progress
GET	<code>/api/rewards/missions</code>	Authoritative mission state
GET	<code>/api/rewards/referrals</code>	Referral code, summary, and history
GET	<code>/api/rewards/loyalty</code>	Current tier, progress, benefits, ladder

Rewards administration

Administration routes under `/api/admin/rewards` cover rules, global configuration, daily rewards, achievements, missions, referral configuration/history, loyalty tiers, user search/detail, occasion unlocks, and reason-recorded adjustments. They call `checkApiPerm("rewards", "view" | "edit")` rather than creating a separate authorisation mechanism.

08 Validation and Error Handling

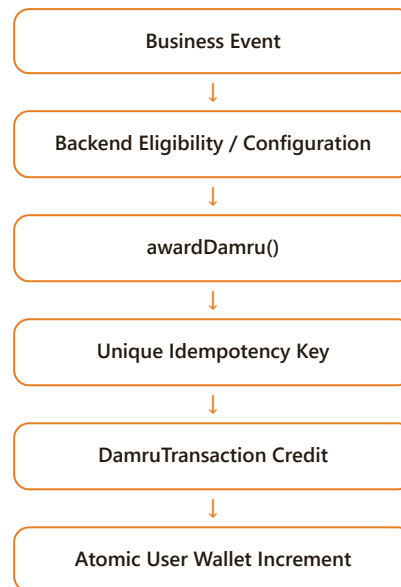
The layered checks every request passes through.

1. Route handlers validate request shapes, numeric ranges, enum membership, dates, and object identity.
2. Authentication helpers establish customer/admin identity.
3. Permission helpers authorise administrative capabilities.
4. Business services recompute eligibility and do not trust frontend progress.
5. Mongoose schemas enforce required values, enums, bounds, and unique indexes.
6. Conditional atomic updates and unique idempotency keys protect financial operations.

Route handlers generally return JSON errors with appropriate `400`, `401`, `403`, `404`, or `500` status codes. Website and mobile callers show safe messages. TanStack Query retries transient failures once but avoids retrying 4xx failures.

09 Damru Reward Architecture

The single controlled path every Damru credit or debit travels.



`lib/rewardEngine.ts` is the central issuer. `awardDamru()` creates the ledger record, atomically increments `damruBalance` and `damruTotalEarned`, patches `balanceAfter`, optionally creates a private coupon, and attempts a notification email. Duplicate transaction keys return a duplicate result without another balance change.

`redeemDamru()` validates the order and redemption policy, conditionally decrements the wallet, and writes an order-bound debit.

`adjustDamru()` records the acting administrator and required reason.

Never introduce a direct wallet write

Direct wallet writes must not be introduced outside these controlled primitives — every new reward-producing feature must call

`awardDamru()`, `redeemDamru()`, or `adjustDamru()`.

10 Reward Event Map

Which trusted backend events fan out into which reward effects.

Registration

- └ Welcome reward
- └ Referral registration association

Login / authenticated activity

- └ Daily login reward
- └ Streak update
- └ Streak achievement evaluation
- └ Streak mission evaluation

Profile activity

- └ Profile achievement evaluation
- └ Profile mission evaluation

Order delivered

- └ First-order reward
- └ Order/spend achievements
- └ Order/spend missions
- └ Referral qualification
- └ Loyalty tier evaluation and optional upgrade bonus

11 Daily Streaks

Server-authoritative streak calculation and reward issuance.

`lib/dailyStreak.ts` contains pure date/projection logic. `lib/getDailyStreakConfig.ts` loads normalised configuration.

`checkAndAwardDailyLogin()` uses a server date string, a conditional user update, and a `daily-login:{userId}:{date}` transaction key.

Cycle behavior and grace-period decisions remain fully admin-configurable rather than hardcoded.

lib/dailyStreak.ts

Pure streak projection and reward-day resolution logic.

lib/getDailyStreakConfig.ts

Loads and normalises the admin-configured day-reward schedule.

Idempotency key

`daily-login:{userId}:{date}` — one credit per eligible calendar day.

12 Achievements

Long-term progress milestones evaluated by a shared engine.

`lib/achievementEngine.ts` resolves progress for `ORDER_COUNT`, `LIFETIME_SPEND`, `LOGIN_STREAK`, `PROFILE_COMPLETE`, and `ACCOUNT_AGE_DAYS` conditions. Definitions are stored in `Achievement`; per-user state is stored in `UserAchievement`. Unlock rewards use the central reward engine and a deterministic idempotency identity, so re-evaluating an already-unlocked achievement never double-pays.

13 Missions

Time-bound challenges with event-level deduplication.

`lib/missionPeriod.ts` calculates period boundaries. `lib/missionEngine.ts` updates applicable missions for `ORDER_COUNT`, `SPENDING_AMOUNT`, `LOGIN_STREAK`, and `PROFILE_COMPLETE` conditions. `MissionEvent` prevents a repeated source event from incrementing progress twice — this is a separate dedup layer from mission-completion idempotency itself.

Supported periods: `ONE_TIME` `DAILY` `WEEKLY` `MONTHLY` `CAMPAIGN`

14 Referrals

Code generation, qualification, and dual-sided reward issuance.

`lib/referralEngine.ts`, `lib/referralCode.ts`, `ReferralConfig`, and `Referral` together implement code generation, registration association, first eligible/delivered-order qualification, referral limits, verification checks, reward delay policy, and duplicate-safe reward issuance for both sides of a referral. Referral API output masks or omits sensitive referred-user information.

15 Loyalty Tiers

Configurable tier ladder with trusted, backend-computed qualification.

`LoyaltyTier` stores name, code, rank, qualification type, min/max range, multiplier, benefits, badge, an optional upgrade bonus, and an activation flag. Active tiers must share a single qualification type; invalid and overlapping ranges are rejected at the model/admin layer. Only the highest tier may omit a maximum.

`lib/loyaltyEngine.ts` loads the active ladder, calculates trusted qualification from delivered orders or lifetime Damru totals, resolves the current and next tier, returns progress and benefits, persists tier identity on the user, and optionally issues an upgrade bonus through `awardDamru()` with the idempotency key `loyalty-tier:{userId}:{tierId}`.

Multipliers are centralised in `getLoyaltyRewardMultiplier()` and currently apply only when the caller explicitly identifies `ORDER_REWARD`. `applyLoyaltyMultiplier()` uses integer rounding. Birthday, anniversary, login, achievement, mission, referral, admin-adjustment, and redemption transactions are **not** multiplied by default.

Reconciliation script

`scripts/recalculate-loyalty-tiers.ts` is rerunnable, reports only aggregate counts, and deliberately sets `issueBonus: false` so re-running it never re-pays upgrade bonuses.

Known debugging note

A standalone loyalty helper runner was observed to block/hang during Mongoose module initialisation in this execution environment during documentation research; no database write occurred. Investigate module-level side effects if a future script exhibits the same symptom.

16 Checkout and Order Lifecycle

From cart to delivery, and the trusted transition that triggers rewards.



Backend routes validate ownership and recompute sensitive totals server-side. Order states are `pending`, `confirmed`, `preparing`, `out_for_delivery`, `delivered`, and `cancelled`; payment states are `pending`, `paid`, and `failed`.

The delivered transition is the trusted reward trigger

The delivered transition in `app/actions/orders.ts` invokes the reward engines (first-order, achievements, missions, referral qualification, loyalty evaluation). Individual engines remain idempotent so a replayed transition never duplicates a financial reward.

17 Website Architecture

App Router structure and where rewards UI lives.

The App Router website is under `app/(website)`. Customer rewards are integrated into `my-profile/page.tsx` and supported by `lib/rewards` types, API wrappers, provider state, and analytics event names. CSS lives under `styles/website`. Client components handle browser interaction while route handlers and server actions retain protected, server-only logic.

18 Mobile Architecture

Expo Router structure and the shared-API pattern.

The Expo app uses `mobile-app/src/app` for routes. `services/rewardsApi.ts` wraps the shared reward endpoints; `types/rewards.ts` defines response contracts; `queryClient.ts` owns retry/cache defaults. Rewards include dashboard/profile integration plus dedicated history, achievement, mission, referral, and loyalty screens, each with loading, retry, error, and pull-to-refresh handling.

19 Admin Architecture and Permissions

One dashboard, module-level permission gating.

Admin routes and pages are grouped under `app/admin`. The rewards page contains tabs for reward rules, Damru config, daily rewards, achievements, missions, referrals, loyalty tiers, and individual customer rewards. `getAdminPerms()` supports server actions and `checkApiPerm()` protects route handlers. Manager role/permission editing reuses the same permission model.

Known permission gap

`role === "admin"` bypasses all permission checks in the current implementation — only the `moderator` role is actually restricted by granular permissions. `checkApiPerm` also has no null-check on the admin lookup, and an inactive admin (`isActive: false`) is never enforced at the permission layer. These are documented findings, not yet remediated.

20 Email and Scheduled Work

Transactional messages and the rewards scheduler.

`lib/email.ts` centralises transactional email behaviour over Nodemailer/SMTP. The rewards scheduler endpoint is `/api/internal/rewards/run-scheduler`; its deployment schedule is configured in `vercel.json`. Scheduler access must remain secret-protected. The code supports verification/reset and reward-related message flows without placing provider secrets in client code.

21 Security Properties

What's already enforced, confirmed directly from the code.

Hashed passwords

Protected verification/token material, never stored or returned in readable form.

Authenticated APIs

Customer endpoints require a valid session and enforce ownership checks.

Permission-gated admin

Administrative routes are checked against assigned module permissions.

Reward idempotency

Unique keys guarantee a reward can never be issued twice for the same event.

Atomic wallet updates

Balance changes use conditional atomic operations with full ledger history.

Order-bound redemption

Damru redemption is tied to a specific, owned order and re-validated server-side.

Private coupon isolation

Customer-specific coupons are never visible to another customer.

Backend-authoritative progress

Reward and loyalty progress is always computed server-side, never trusted from a client.

Safe reconciliation output

Reconciliation scripts report aggregate counts only, no personal data.

Known gaps (documented, not remediated in this pass)

No rate limiting exists anywhere in the current implementation. OTP values are carried inside a signed-but-not-encrypted JWT rather than being encrypted at rest. Coupon `perUserLimit` / `usedBy` fields exist on the schema but are not enforced or updated by any route, and coupon `usageLimit` has a TOCTOU race under concurrent redemption. See also the admin permission gap noted in Section 19.

22 Environment Variables

Expected configuration categories — names only, never values.

Never copy values into documentation

Variable names must be taken from code and deployment configuration before production setup. Never copy values from `.env.local` into documentation, commits, logs, or client bundles.

Expected categories, based on how the codebase consumes configuration:

- MongoDB connection string
- Authentication / session secrets
- Public application URL
- Mobile API base URL
- Email provider credentials (SMTP)
- Scheduler protection secret (for `/api/internal/rewards/run-scheduler`)
- Any payment provider credentials used by the deployed environment

23 Local Development

Getting the website, backend, and mobile app running locally.

```
# Website/backend
npm install
npm run dev

# Static validation
npx tsc --noEmit
npm run lint

# Mobile
cd mobile-app
npm install
npm start
npm run android
npm run typecheck
```

Configure required environment variables locally before starting. The root `seed` script runs `npx tsx scripts/seed.ts`. Do not point seed or reconciliation commands at production without explicit operational review.

24 Build and Deployment

What's confirmed about how this ships.

The website uses `npm run build` and `npm start`. `vercel.json` supplies the discovered deployment scheduling configuration; hosting assumptions beyond committed configuration are not confirmed. Expo/EAS configuration under `mobile-app` governs native build profiles.

Production requires: a reachable API URL, database, authentication secrets, email setup, scheduler secret, configured admin permissions, and seeded reward/tier configuration.

25 Testing and Known Issues

Confirmed validation status as of 7 August 2026.

Check	Result
Root <code>npx tsc --noEmit</code>	Passes
Mobile <code>npx tsc --noEmit</code>	Passes
Targeted lint	Existing strict-rule issues in large rewards/profile files and admin handlers
Mobile Expo lint	Known <code>eslint-config-expo/flat</code> environment issue
Database-backed integration test harness	Not found
Automated test infrastructure (general)	Not found anywhere in the repository
CI/CD pipeline	Not found

No unrelated dependency was installed to mask the Expo lint issue

A pure loyalty helper runner was also observed to block/hang during Mongoose module initialisation in the current execution environment; no database write occurred as a result.

26 Common Extension Workflows

Repeatable patterns for extending the platform safely.

Add an API

Create a route handler under `app/api`, authenticate first, validate input, call an existing service/model pattern, return safe JSON, and add web/mobile types where consumed.

Add a reward safely

1. Define the trusted backend trigger.
2. Load server-side configuration and validate eligibility.
3. Generate a deterministic idempotency key.
4. Call `awardDamru()`; never increment balance directly.
5. Attach useful non-sensitive transaction metadata.
6. Add administration configuration only if required.
7. Extend API and client display types.
8. Test the same event twice.

Add an achievement or mission

Extend the model enum and engine calculation, integrate only at an existing trusted event, add admin/customer labels, and verify progress plus duplicate-event behaviour.

Add a mobile rewards screen

Add an Expo Router file, a typed service function, loading/error/retry/refresh states, query invalidation where mutations occur, and a navigation entry from the rewards profile section.

27 Troubleshooting

Symptom-first lookup for common issues.

Symptom	Checks
Database connection failure	Confirm Mongo URI name/value locally, network access, and database availability
Customer appears logged out	Inspect customer session cookie/token expiry and <code>/api/user/me</code> response
Admin receives forbidden response	Inspect assigned rewards/resource permissions and the <code>checkApiPerm</code> action
Damru not credited	Inspect rule activation, event eligibility, idempotency key, and ledger record
Duplicate reward skipped	Expected when the same deterministic key already exists
Loyalty tier absent	Configure at least one valid active tier and run reconciliation for existing users
Wrong loyalty progress	Confirm all active tiers share qualification type and delivered-order data is correct
Coupon unavailable	Check activation window, usage limits, ownership, and minimum order value
Mobile API failure	Confirm mobile API base URL, device network reachability, and authentication state
Expo lint failure	Verify the known <code>eslint-config-expo/flat</code> environment issue before changing app code

28 Production Checklist

Confirm before pointing this at real traffic.

- ☐ Production environment variable names and values configured securely
- ☐ Production MongoDB indexes created and reviewed
- ☐ Customer and admin authentication verified
- ☐ Email sender/domain verified
- ☐ Payment behaviour tested with production-safe credentials
- ☐ Reward rules, streaks, achievements, missions, referrals, and loyalty tiers reviewed
- ☐ Scheduler endpoint protected and schedule verified
- ☐ Admin permissions assigned by least privilege
- ☐ Website build and both TypeScript checks pass
- ☐ Mobile API URL and native build profiles verified
- ☐ Reward replay/idempotency scenarios tested against a staging database
- ☐ Backup, monitoring, and incident ownership established

29 Developer Onboarding Checklist

A first week on this codebase.

- ☐ Clone and install root/mobile dependencies
- ☐ Configure local environment without committing secrets
- ☐ Run website and mobile applications
- ☐ Test customer and admin login
- ☐ Review `Order`, `User`, `DamruTransaction`, and `rewardEngine.ts`
- ☐ Trace the delivered-order reward event map
- ☐ Review admin permission checks
- ☐ Run root and mobile TypeScript
- ☐ Read known lint/test limitations
- ☐ Use deterministic keys for every new reward-producing event

This document describes confirmed repository structure and implementation as of 7 August 2026. Where operational deployment details are not committed, they are explicitly not assumed. No secret values or customer records are included.